Indeksy, optymalizator
Lab1

Imiona i nazwiska: Karolina Węgrzyn, Patrycja Markiewicz

Celem ćwiczenia jest zapoznanie się z planami wykonania zapytań (execution plans), oraz z budową i możliwością wykorzystaniem indeksów.

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

> Wyniki:

```
--  ...
```

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki

- dołącz kod rozwiązania w formie tekstowej/źródłowej
- najlepiej plik .md
  - ewentualnie sql

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie

- MS SQL Server
- SSMS - SQL Server Management Studio
  - ewentualnie inne narzędzie umożliwiające komunikację z MS SQL Server i analizę planów zapytań
- przykładowa baza danych AdventureWorks2017.

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Przygotowanie

Stwórz swoją bazę danych o nazwie lab1.

```
create database lab1
go

use lab1
go
```

Część 1

Celem tej części ćwiczenia jest zapoznanie się z planami wykonania zapytań (execution plans) oraz narzędziem do automatycznego generowania indeksów.

Dokumentacja/Literatura

Przydatne materiały/dokumentacja. Proszę zapoznać się z dokumentacją:

- https://docs.microsoft.com/en-us/sql/tools/dta/tutorial-database-engine-tuning-advisor
- https://docs.microsoft.com/en-us/sql/relational-databases/performance/start-and-use-the-database-engine-tuning-advisor
- https://www.simple-talk.com/sql/performance/index-selection-and-the-query-optimizer
- https://blog.quest.com/sql-server-execution-plan-what-is-it-and-how-does-it-help-with-performance-problems/

Operatory (oraz reprezentujące je piktogramy/Ikonki) używane w graficznej prezentacji planu zapytania opisane są tutaj:

- https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference

Wykonaj poniższy skrypt, aby przygotować dane:

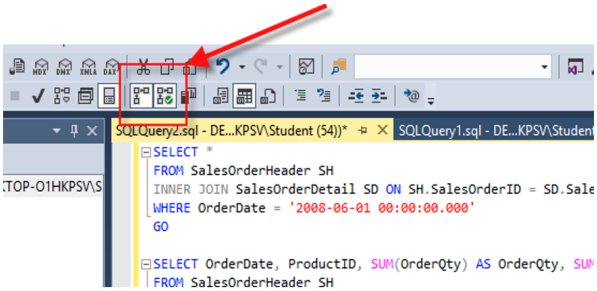
```
select * into [salesorderheader]
from [adventureworks2017].sales.[salesorderheader]
go

select * into [salesorderdetail]
from [adventureworks2017].sales.[salesorderdetail]
go
```

Zadanie 1 - Obserwacja

Wpisz do MSSQL Managment Studio (na razie nie wykonuj tych zapytań):

```
-- zapytanie 1
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go

-- zapytanie 1.1
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2013-01-28 00:00:00.000'
go

-- zapytanie 2
select orderdate, productid, sum(orderqty) as orderqty,
       sum(unitpricediscount) as unitpricediscount, sum(linetotal)
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go

-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01','2008-06-02', '2008-06-03', '2008-06-04', '2008-06-05')
go

-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```

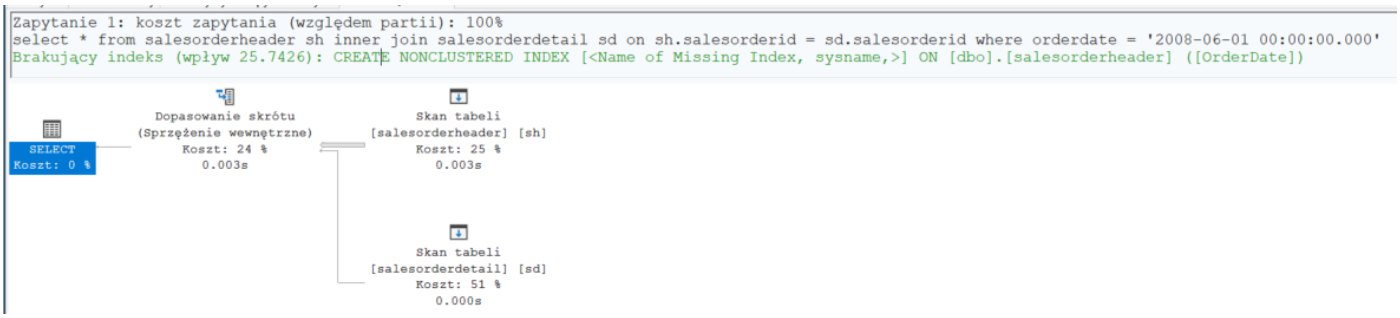
Włącz dwie opcje: **Include Actual Execution Plan** oraz **Include Live Query Statistics**:

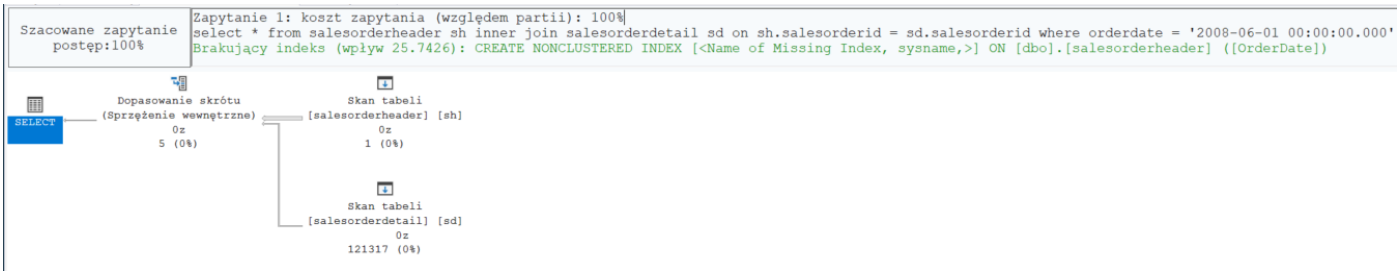


Teraz wykonaj poszczególne zapytania (najlepiej każde analizuj oddzielnie). Co można o nich powiedzieć? Co sprawdzają? Jak można je zoptymalizować?

> Wyniki:

```
-- zapytanie 1
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go
```





Co sprawdza to zapytanie?

Zapytanie pobiera wszystkie dane ze złączonych za pomocą ID zamówienia tabel salesorderheader i salesorderdetail dla zamówień z dnia 2008-06-01.

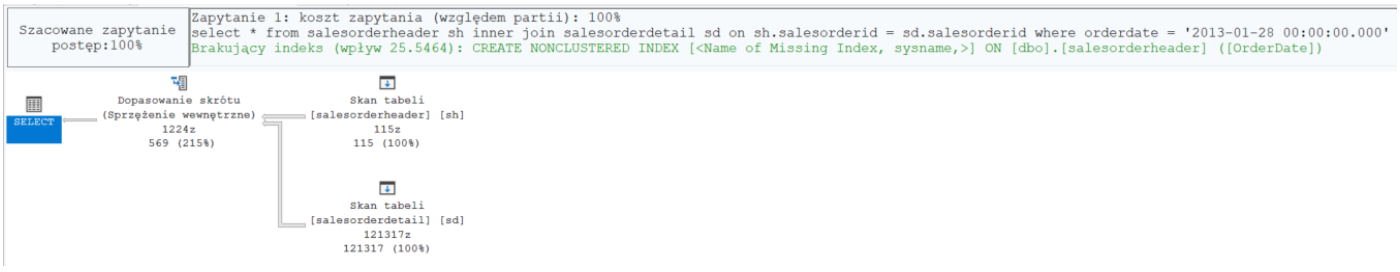
Co można o nim powiedzieć?

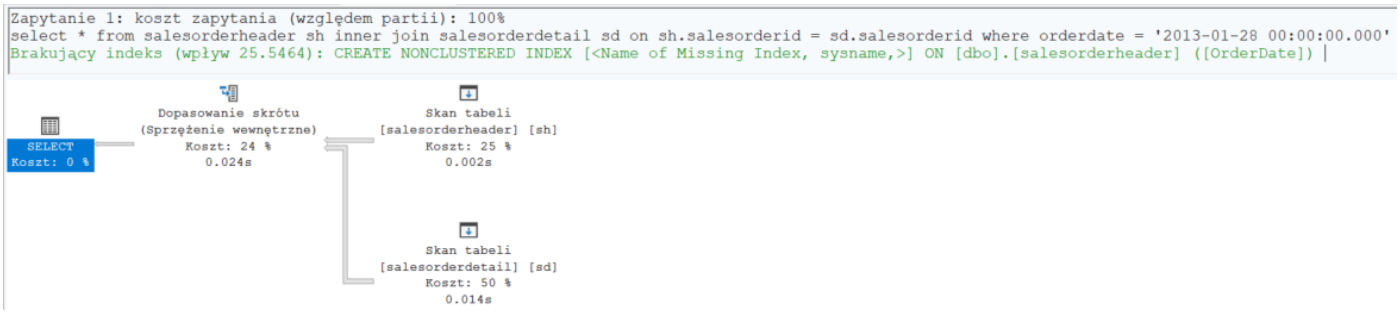
Z planu zapytania wynika, że obie tabele nie mają zdefiniowanych żadnych indeksów ani kluczy głównych. Baza danych używa Table Scan, zmuszona do sekwencyjnego czytania całych tabel. Analiza kosztów pokazuje, że najdroższą operacją jest skanowanie tabeli salesorderdetail, co pochłania aż 51% całkowitego kosztu zapytania i zmusza silnik do przetworzenia ponad 121 tysięcy wierszy. Skanowanie tabeli salesorderheader to kolejne 25% kosztu. Do połączenia tych zbiorów wykorzystano operator Hash Match, co generuje pozostałe 24% kosztu zapytania.

Jak można je zoptymalizować?

Sam optymalizator sugeruje utworzenie indeksu nieklastrowanego na kolumnie OrderDate w tabeli salesorderheader, co według jego szacunków poprawiłoby wydajność o ponad 25%.

```
-- zapytanie 1.1
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2013-01-28 00:00:00.000'
go
```





Co sprawdza to zapytanie?

To samo co dla zapytania 1 ale dla zamówień z dnia 2013-01-28.

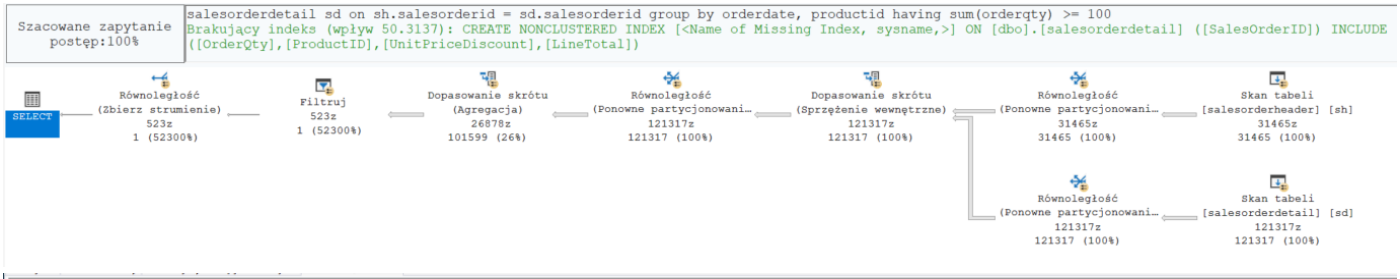
Co można o nim powiedzieć?

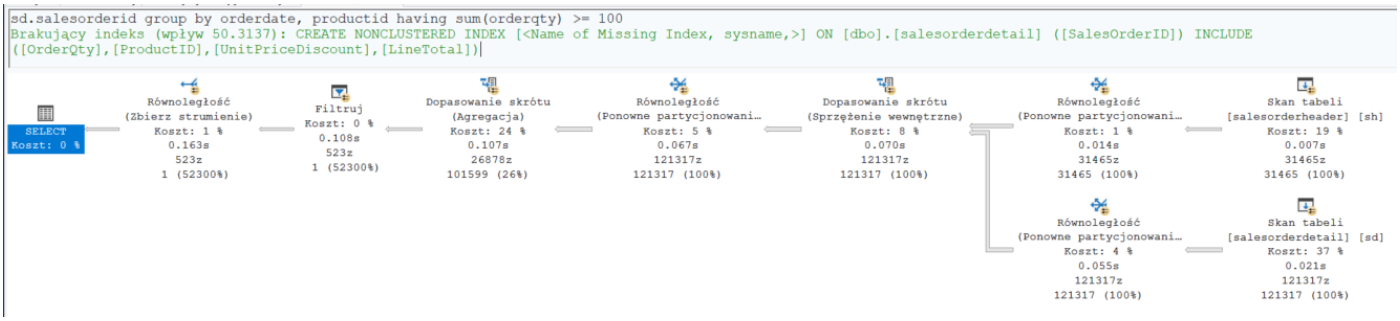
Plan wykonania jest niemal identyczny jak w poprzednim zapytaniu. Obie tabele nie mają indeksów, co wymusza użycie operatorów Table Scan. Skanowanie tabeli salesorderdetail pochłania 50% kosztu zapytania, a skanowanie salesorderheader 25%. Zbiory połączono za pomocą operatora Hash Match generując 24% kosztu. Ciekawostką ze statystyk na żywo jest to, że optymalizator niedoszacował liczby wynikowych wierszy przy łączeniu, gdyz zwrócono 1224 wiersze z 569 szacowanych, co daje 215% normy.

Jak można je zoptymalizować?

Identycznie jak w poprzednim przypadku optymalizator sugeruje utworzenie indeksu nieklastrowanego na kolumnie OrderDate w tabeli salesorderheader. Szacowany spadek kosztu całego zapytania po dodaniu tego indeksu to ponad 25%.

```
-- zapytanie 2
select orderdate, productid, sum(orderqty) as orderqty,
       sum(unitpricediscount) as unitpricediscount, sum(linetotal)
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go
```





Co sprawdza to zapytanie?

Zapytanie łączy tabele salesorderheader i salesorderdetail, grupuje dane po dacie zamówienia `orderdate` i ID produktu `productid`, a następnie oblicza sumy dla ilości `orderqty`, zniżek `unitpricediscount` i wartości całkowitej `linetotal`. Wynik jest filtrowany i zwraca tylko te grupy, dla których suma zamowionych sztuk wynosi co najmniej 100.

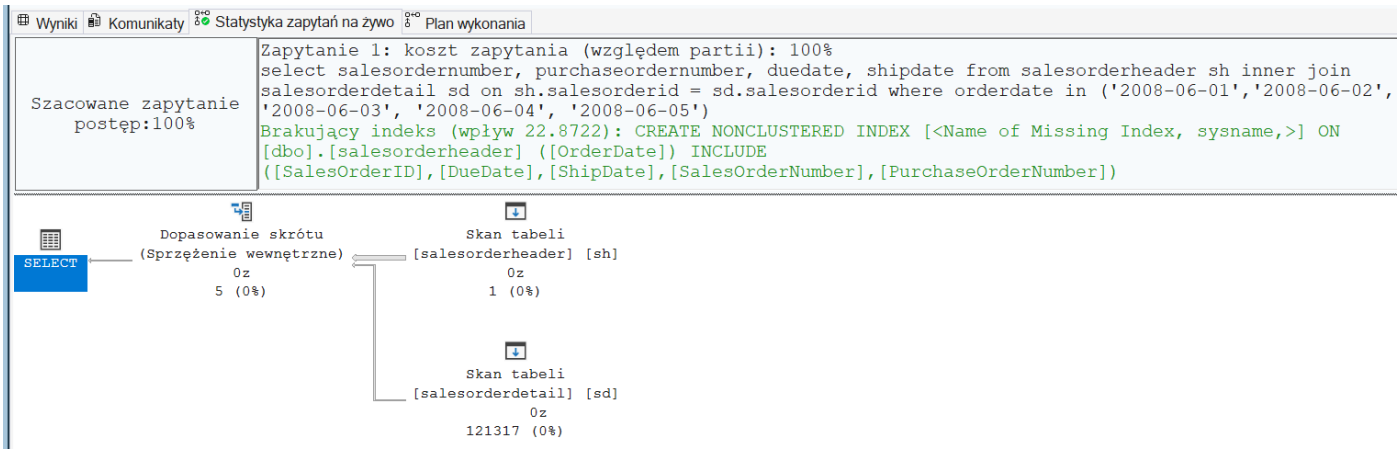
Co można o nim powiedzieć?

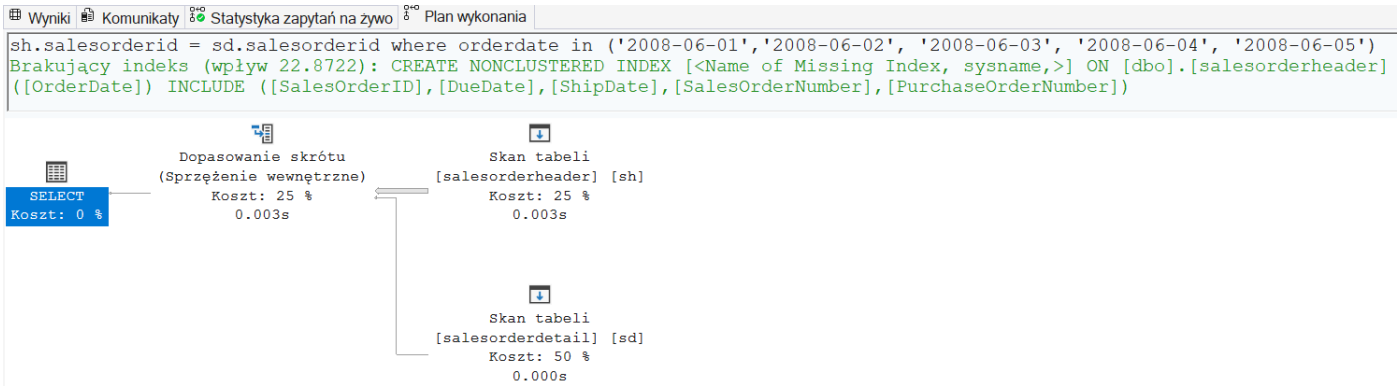
Z planu wynika, że ze względu na brak indeksów, silnik ponownie wykonuje Table Scany. Generuje to największe koszty, 37% dla salesorderdetail oraz 19% dla salesorderheader. Tabele połączono za pomocą operatora Hash Match generując 8% kosztu. Do pogrupowania danych i obliczenia sum użyto kolejnego operatora Hash Match (agregacja) generujac 24% kosztu. Na końcu operator Filtruj aplikuje warunek HAVING. Warto zauważyć ogromne niedoszacowanie na końcu planu, silnik spodziewał się zwrócić 1 wiersz, a zwrócił 523.

Jak można je zoptymalizować?

Optymalizator sugeruje utworzenie indeksu nieklastrowanego na tabeli salesorderdetail. Kluczem tego indeksu ma być kolumna SalesOrderID, a w klauzuli INCLUDE powinny znaleźć się następujące kolumny: OrderQty, ProductID, UnitPriceDiscount oraz LineTotal. Według wyliczeń optymalizatora, dodanie tej struktury obniży koszt zapytania o ponad 50%.

```
-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01','2008-06-02', '2008-06-03', '2008-06-04', '2008-06-05')
go
```





Co sprawdza to zapytanie?

Zapytanie pobiera numer zamówienia, numer zakupu, datę wymaganą i datę wysyłki ze złączonych za pomocą ID zamówienia tabel salesorderheader i salesorderdetail. Wyniki są zawężone tylko do zamówień złożonych w ciągu pięciu konkretnych dni na początku czerwca 2008 roku.

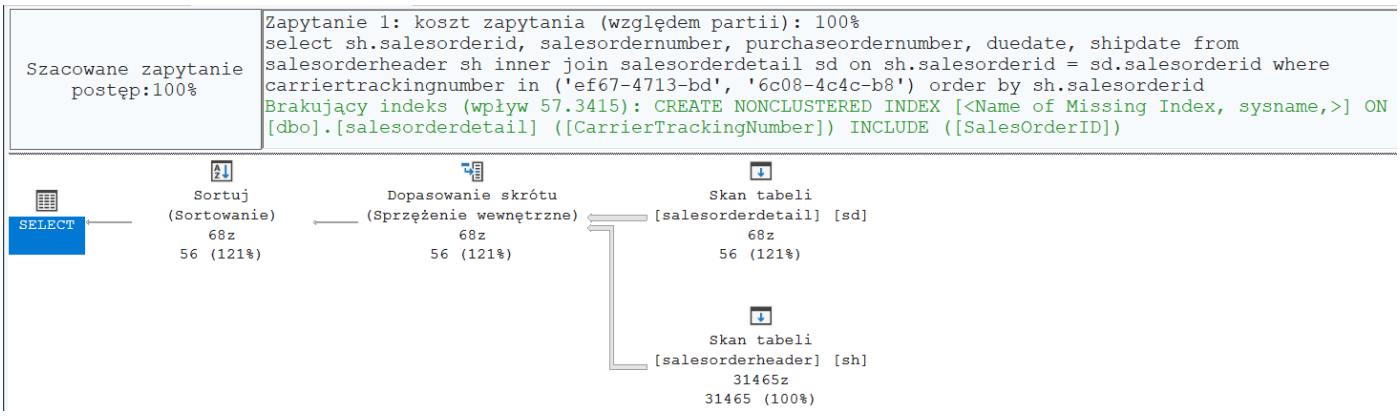
Co można o nim powiedzieć?

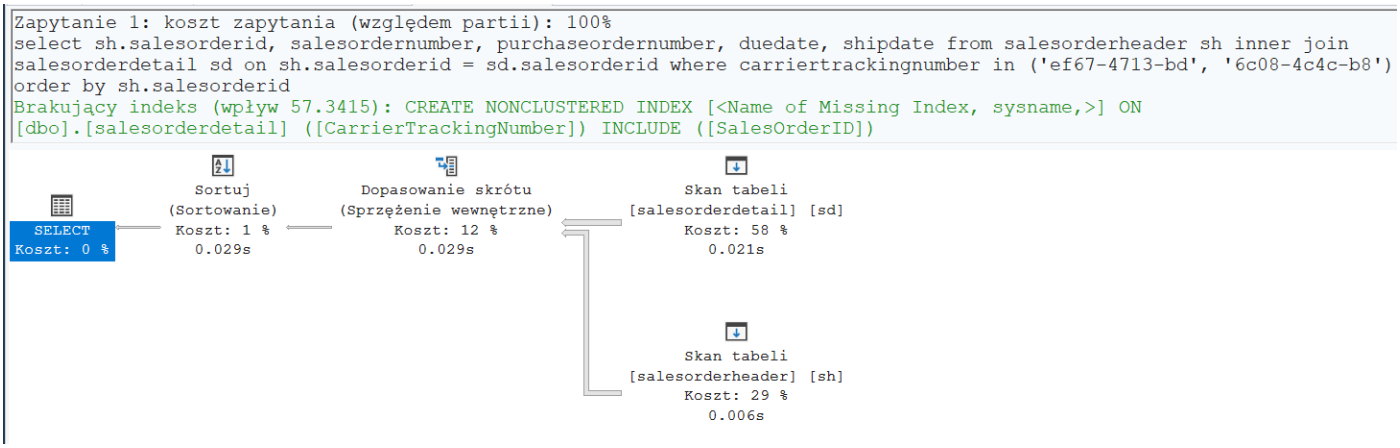
Obie tabele nie posiadają indeksów, co wymusza operacje Table Scan. Skanowanie tabeli salesorderdetail generuje 50% kosztów, a tabeli salesorderheader kolejne 25%. Baza łączy te zbiory za pomocą operatora Hash Match, co pochłania pozostałe 25% zasobów całego zapytania.

Jak można je zoptymalizować?

Optymalizator sugeruje utworzenie indeksu nieklastrowanego na tabeli salesorderheader. Głównym kluczem wyszukiwania ma być kolumna użyta w warunku WHERE `OrderDate`. Optymalizator zaleca wrzucenie do klauzuli INCLUDE wszystkich pozostałych kolumn użytych w zapytaniu: SalesOrderID, DueDate, ShipDate, SalesOrderNumber i PurchaseOrderNumber. Według szacunków, dodanie takiego indeksu obniży koszt zapytania o niecałe 23%.

```
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```





Co sprawdza to zapytanie?

Zapytanie pobiera podstawowe informacje o zamówieniach, łącząc tabele salesorderheader i salesorderdetail. Wyniki są mocno odfiltrowane, szukamy tylko konkretnych paczek o dwóch podanych numerach przewozowych. Na sam koniec wynik jest sortowany rosnąco po ID zamówienia.

Co można o nim powiedzieć?

Z planu znów wynika spory problem wydajnościowy, najdroższą operacją (58% kosztu) jest skanowanie całej tabeli salesorderdetail tylko po to, by znaleźć w niej 68 wierszy pasujących do podanych numerów przewozowych. Następnie baza skanuje tabelę salesorderheader generując 29% kosztu i łączy to wszystko za pomocą Hash Match 12% kosztu. Operacja sortowania na samym końcu kosztuje zaledwie 1%, ponieważ do posortowania została już tylko garstka odfiltrowanych danych.

Jak można je zoptymalizować?

Optymalizator sugeruje utworzenie indeksu nieklastrowanego na tabeli salesorderdetail, na kolumnie CarrierTrackingNumber. Z kolei do klauzuli INCLUDE optymalizator każe dorzucić SalesOrderID, dzięki temu, gdy baza znajdzie już odpowiedni numer paczki, od razu będzie miała pod ręką ID potrzebne do zrobienia joina z drugą tabelą, bez dodatkowego skakania po dysku. Szacowany zysk z tej optymalizacji to ponad 57%.

Zadanie 2 - Dobór indeksów / optymalizacja

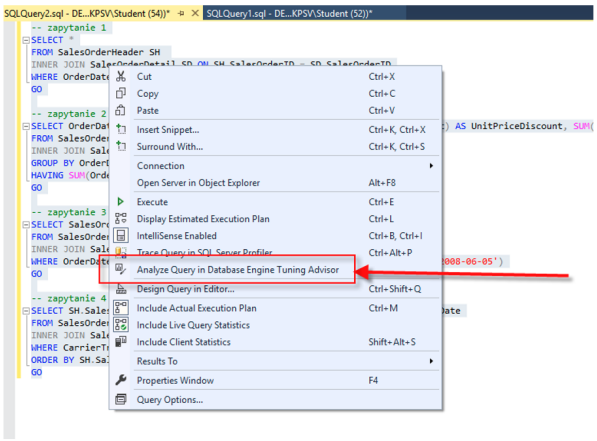
Do wykonania tego ćwiczenia potrzebne jest narzędzie SSMS

Zapytania 1, 2, 3, 4 z poprzedniego zadania

```
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go

-- zapytanie 2
select orderdate, productid, sum(orderqty) as orderqty,
       sum(unitpricediscount) as unitpricediscount, sum(linetotal)
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go

-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01','2008-06-02', '2008-06-03', '2008-06-04', '2008-06-05')
go

-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
```

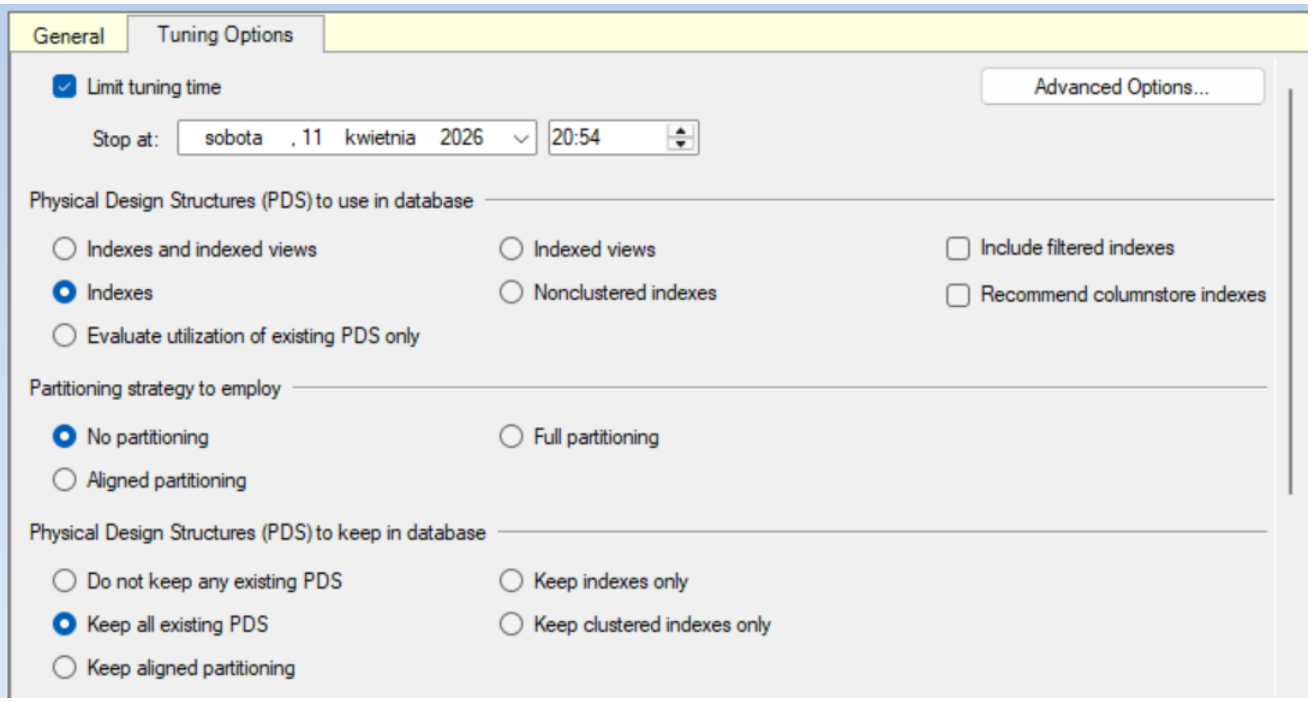
```
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```

Zaznacz wszystkie zapytania, i uruchom je w **Database Engine Tuning Advisor**:



Sprawdź zakładkę **Tuning Options**, co tam można skonfigurować?

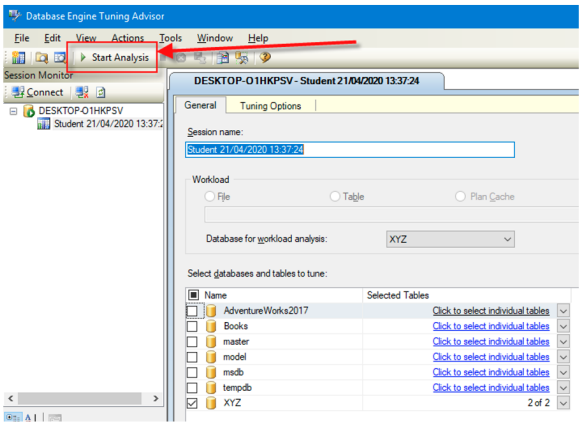
Wyniki:



W zakładce Tuning Options można określić granice działania DTA. Główne parametry, które można tam skonfigurować to:

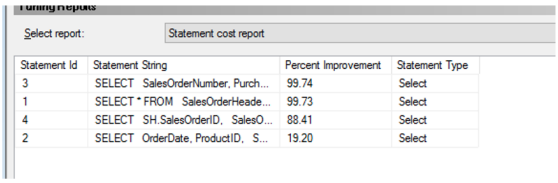
- **Limit tuning time** pozwala określić maksymalny czas, jaki doradca ma poświęcić na analizę zapytania.
- **Physical Design Structures to use in database** określa jakich narzędzi doradca może użyć do poprawy wydajności - u nas zaznaczone są indeksy.
- **Partitioning strategy to employ** pozwala na fizyczny podział tabel na mniejsze fragmenty - domyślnie ustawiony jest brak podziału.
- **Physical Design Structures to keep in database** decyduje o losie obecnych obiektów w bazie. Domyślnie wybrana była opcja `Keep all existing PDS`, aby doradca jedynie sugerował nowe indeksy, bez usuwania tych utworzonych wcześniej.

Użyj **Start Analysis**:



Zaobserwuj wyniki w **Recommendations**.

Przejdź do zakładki **Reports**. Sprawdź poszczególne raporty. Główną uwagę zwróć na koszty i ich poprawę:

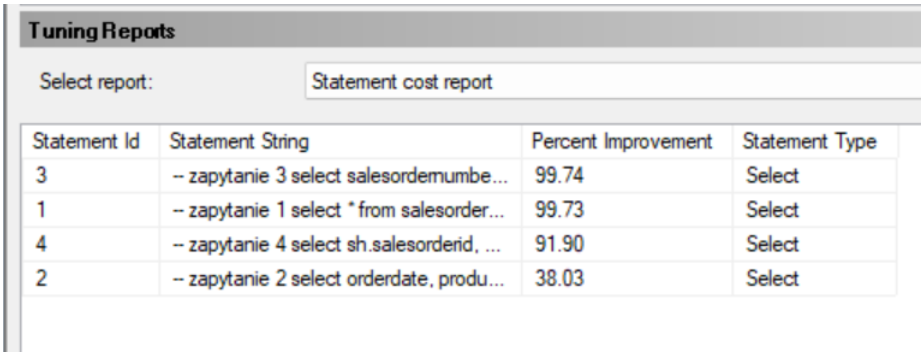


Zapisz poszczególne rekomendacje:

Uruchom zapisany skrypt w Management Studio.

Opisz, dlaczego dane indeksy zostały zaproponowane do zapytań:

> Wyniki:



Tuning Reports

Select report: Statement cost report

Statement Id	Statement String	Percent Improvement	Statement Type
3	-- zapytanie 3 select salesordernumbe...	99.74	Select
1	-- zapytanie 1 select * from salesorder...	99.73	Select
4	-- zapytanie 4 select sh.salesorderid, ...	91.90	Select
2	-- zapytanie 2 select orderdate, produ...	38.03	Select

Raport kosztów wskazuje na drastyczny wzrost wydajności po zastosowaniu rekomendacji. **Zapytania 1 i 3** zostały zoptymalizowane niemal perfekcyjnie, osiągając poprawę na poziomie blisko 100%. **Zapytanie 4** przyspieszyło bardzo znacząco, bo o 91.90%. **Zapytanie 2** zanotowało najmniejszy, choć wciąż istotny skok wydajności o 38.03%. Mniejsza poprawa zapewne wynika z faktu, że zapytanie to wykorzystuje funkcje agregujące oraz grupowanie.

```
CREATE NONCLUSTERED INDEX [_dta_index_salesorderdetail_5_1237579447__K1_2_3_4_5_6_7_8_9_10_11] ON [dbo].[salesorderdetail]
(
    [SalesOrderID] ASC
)
INCLUDE([SalesOrderDetailID],[CarrierTrackingNumber],[OrderQty],[ProductID],[SpecialOfferID],[UnitPrice],[UnitPriceDiscount],[LineTotal],
[rowguid],[ModifiedDate]) WITH (SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF) ON [PRIMARY]
go

CREATE NONCLUSTERED INDEX [_dta_index_salesorderdetail_5_1237579447__K5_1_4_8_9] ON [dbo].[salesorderdetail]
(
    [ProductID] ASC
)
INCLUDE([SalesOrderID],[OrderQty],[UnitPriceDiscount],[LineTotal]) WITH (SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF) ON [PRIMARY]
go

CREATE NONCLUSTERED INDEX [_dta_index_salesorderdetail_5_1237579447__K3_K1] ON [dbo].[salesorderdetail]
(
    [CarrierTrackingNumber] ASC,
    [SalesOrderID] ASC
)WITH (SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF) ON [PRIMARY]
go

CREATE NONCLUSTERED INDEX [_dta_index_salesorderdetail_5_1237579447__K1] ON [dbo].[salesorderdetail]
(
```

```
    [SalesOrderID] ASC
)WITH (SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF) ON [PRIMARY]
go

CREATE NONCLUSTERED INDEX [_dta_index_salesorderheader_5_1221579390__K3_K1_2_4_5_6_7_8_9_10_11_12_13_14_15_16_17_18_19_20_21_22_23_24_25_26] ON
[dbo].[salesorderheader]
(
    [OrderDate] ASC,
    [SalesOrderID] ASC
)
INCLUDE([RevisionNumber],[DueDate],[ShipDate],[Status],[OnlineOrderFlag],[SalesOrderNumber],[PurchaseOrderNumber],[AccountNumber],[CustomerID],
[SalesPersonID],[TerritoryID],[BillToAddressID],[ShipToAddressID],[ShipMethodID],[CreditCardID],[CreditCardApprovalCode],[CurrencyRateID],
[SubTotal],[TaxAmt],[Freight],[TotalDue],[Comment],[rowguid],[ModifiedDate]) WITH (SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF) ON
[PRIMARY]
go

CREATE NONCLUSTERED INDEX [_dta_index_salesorderheader_5_1221579390__K1_4_5_8_9] ON [dbo].[salesorderheader]
(
    [SalesOrderID] ASC
)
INCLUDE([DueDate],[ShipDate],[SalesOrderNumber],[PurchaseOrderNumber]) WITH (SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF) ON
[PRIMARY]
go

CREATE NONCLUSTERED INDEX [_dta_index_salesorderheader_5_1221579390__K3_K1] ON [dbo].[salesorderheader]
(
    [OrderDate] ASC,
    [SalesOrderID] ASC
)WITH (SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF) ON [PRIMARY]
go
```

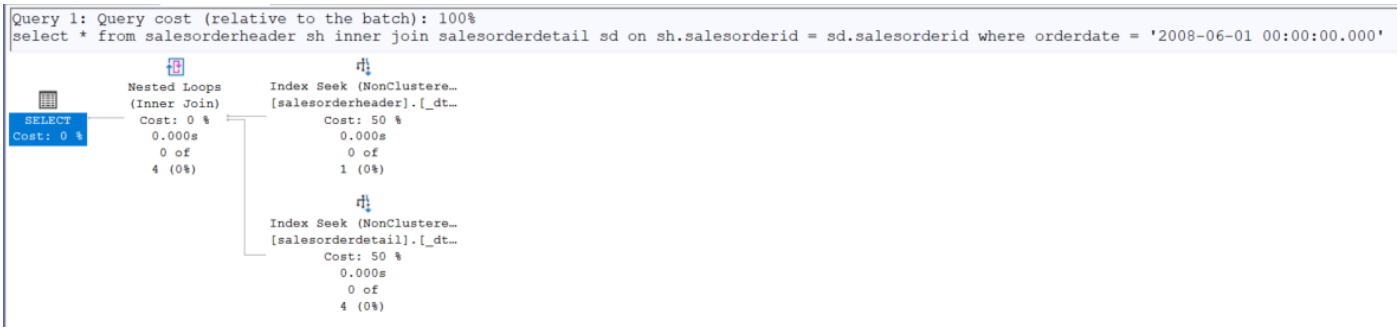
Powyższe indeksy zostały zaproponowane do zapytań z następujących powodów:

- **Optymalizacja złączeń** `join` - każde z czterech zapytań łączy tabele salesorderheader i salesorderdetail po kolumnie SalesOrderID. Advisor oparł większość nowych indeksów właśnie na tym kluczu, co pozwala silnikowi na błyskawiczne dopasowywanie do siebie wierszy z obu tabel.
- **Szybkie filtrowanie i grupowanie** - zapytania wykorzystują klauzule WHERE i GROUP BY na konkretnych kolumnach np. orderdate, carriertrackingnumber, productid. Utworzenie indeksów na tych polach pozwala na użycie wydajnej operacji przeszukiwania jak np Index Seek zamiast czytania całych tabel.
- **Zastosowanie indeksów pokrywających** - doradca rozszerzył indeksy o dodatkowe kolumny wywoływane w poleceniach SELECT np DueDate, ShipDate, LineTotal. Tworzy to tzw. covering index, w którym silnik bazy danych znajduje wszystkie wymagane przez zapytanie informacje bezpośrednio w strukturze indeksu. Eliminuje to konieczność odwoływania się do głównej tabeli bazowej, co bezpośrednio zmniejsza liczbę operacji odczytu.

Sprawdź jak zmieniły się Execution Plany. Opisz zmiany:

> Wyniki:

```
-- zapytanie 1
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go
```



Zamiast skanowania całych tabel, silnik SQL wykorzystał operację Index Seek (50% kosztu) dla obu tabel, pobierając dane bezpośrednio z nowych indeksów pokrywających. Do połączenia odnalezionych rekordów użyto Nested Loops (0% kosztu). Dzięki temu zapytanie całkowicie omija odczyt niepotrzebnych wierszy z dysku, co drastycznie obniża jego całkowity koszt.

```
-- zapytanie 2
select orderdate, productid, sum(orderqty) as orderqty,
       sum(unitpricediscount) as unitpricediscount, sum(linetotal)
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go
```

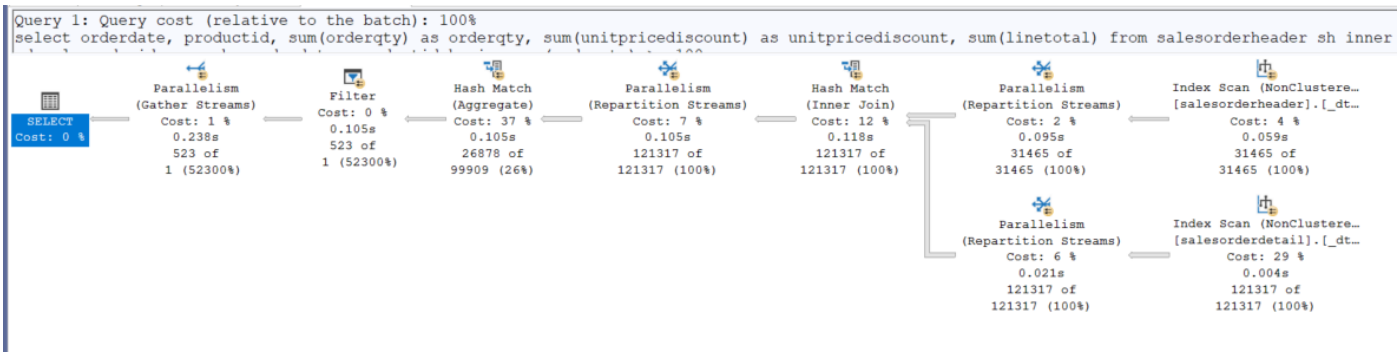


Table Scany zostały zastąpione przez odczyty Index Scan z nowych indeksów. Dalsza struktura planu (Parallelism, Hash Match Join/Aggregate) nie uległa zmianie. Mniejsza poprawa w porównaniu z pozostałymi zapytaniami wynika z faktu, że zapytanie wciąż musi zeskanować sporo danych i wykonać operację grupowania, która sama pochłania aż 37% całkowitego kosztu.

```sql
-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01','2008-06-02', '2008-06-03', '2008-06-04', '2008-06-05')
go
```

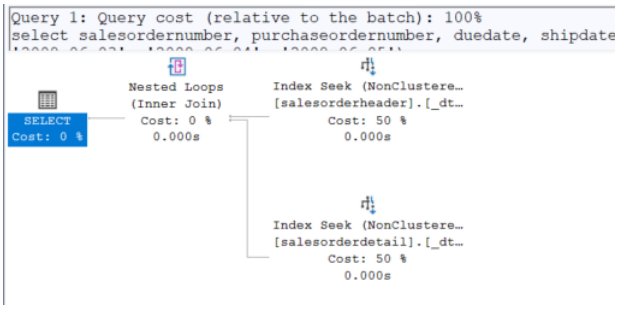


Table Scany zostały wyeliminowane i zastąpione przez szybkie odczyty Index Seek z nowych indeksów. Zmienił się także fizyczny algorytm łączenia tabel z Hash Match na bezkosztowy Nested Loops. Tłumaczy to odnotowaną poprawę wydajności o blisko 100%.

```sql
-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```

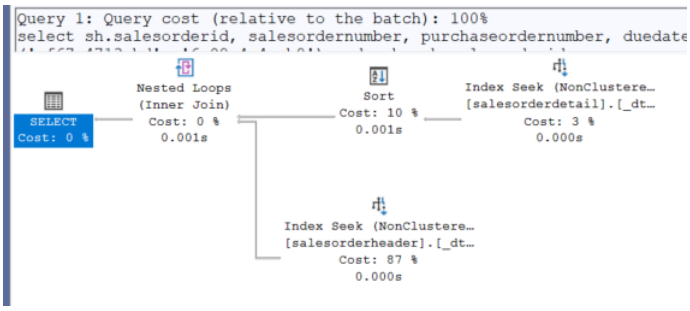


Table Scany na obu tabelach zostały zastąpione przez szybkie odczyty Index Seek z nowych indeksów. Silnik bazy danych zmienił algorytm łączenia tabel z Hash Match na Nested Loops. W planie wykonania uwidoczniła się operacja Sort niezbędna do uporządkowania wstępnie przefiltrowanych rekordów. Dzięki drastycznej redukcji ilości czytanych danych z dysku całkowity koszt zapytania spadł o ponad 91%.

Część 2

Celem ćwiczenia jest zapoznanie się z różnymi rodzajami indeksów oraz możliwością ich wykorzystania

Dokumentacja/Literatura

Przydatne materiały/dokumentacja. Proszę zapoznać się z dokumentacją:

- https://docs.microsoft.com/en-us/sql/relational-databases/indexes/indexes
- https://docs.microsoft.com/en-us/sql/relational-databases/sql-server-index-design-guide

- https://www.simple-talk.com/sql/performance/14-sql-server-indexing-questions-you-were-too-shy-to-ask/
- https://www.sqlshack.com/sql-server-query-execution-plans-examples-select-statement/

Zadanie 3 - Indeksy klastrowane I nieklastrowane

Skopiuj tabelę `Customer` do swojej bazy danych:

```
select * into customer from adventureworks2017.sales.customer
```

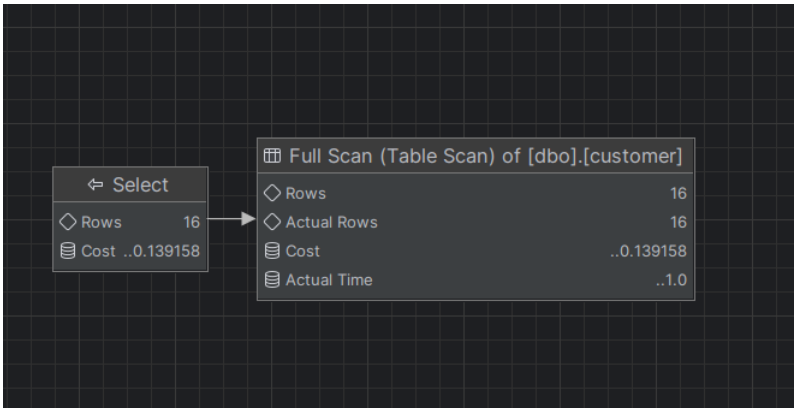
Wykonaj analizy zapytań:

```
--- 1
select * from customer where storeid = 594

--- 2
select * from customer where storeid between 594 and 610
```

Zanotuj czas zapytania oraz jego koszt koszt:

> Wyniki:

Zapytanie	Koszt	Czas (ms)
1	0.139158	1.0
2	0.139158	1.0



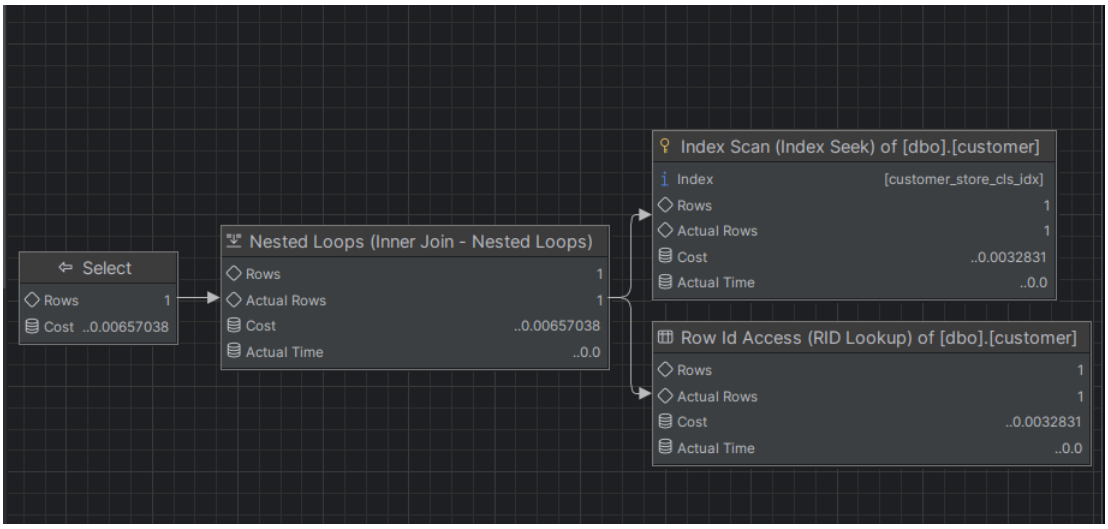
Dodaj indeks:

```
create  index customer_store_cls_idx on customer(storeid)
```

Jak zmienił się plan i czas? Czy jest możliwość optymalizacji?

> Wyniki:

Zapytanie	Koszt	Czas (ms)
1	0.00657038	0.0
2	0.0507122	0.0



Po dodaniu indeksu serwer wykorzystał go do wyszukiwania rekordów, przez co czas i koszt obu zapytań zmalał. Dla 2 zapytania koszt był troche większy, bo serwer musiał odczytać większą ilość rekordów z indeksu. Widać również z planu, że indeks nie zawierał wszystkich kolumn potrzebnych w tym zapytaniu, dlatego reszte danych serwer musiał odczytać bezpośrednio z tabeli.

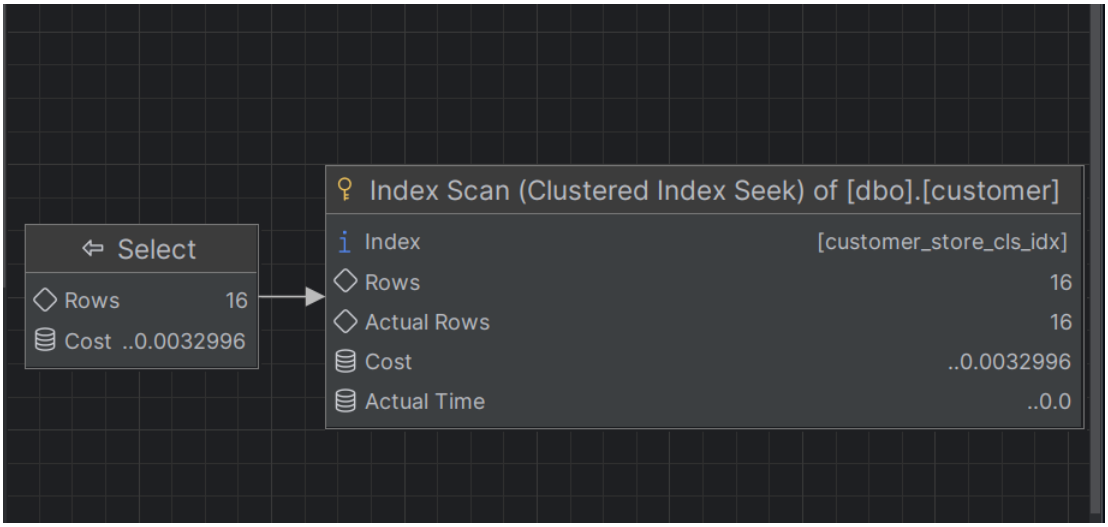
Możliwością optymalizacji jest zastosowanie indeksu klastrowanego, bo serwer bezpośrednio będzie wiedział gdzie jest szukany rekord - uporządkowanie wg kolumny storeid oraz ten typ indeksu zawiera pełne wiersze tabeli.

Dodaj indeks klastrowany:

```
create clustered index customer_store_cls_idx on customer(storeid)
```

Czy zmienił się plan/koszt/czas? Skomentuj dwa podejścia w wyszukiwaniu krotek.

> Wyniki:
>
> | Zapytanie | Koszt | Czas (ms) |
> |---|---|---|
> | 1 | 0.0032831 | 0.0 |
> | 2 | 0.0032996 | 0.0 |



Dla indeksu klastrowanego koszt jeszcze bardziej zmalał - udana optymalizacja. Widać z planu, że wszystkie informacje zostały odczytane bezpośrednio z indeksu.

Zadanie 4 - dodatkowe kolumny w indeksie

Celem zadania jest porównanie indeksów zawierających dodatkowe kolumny.

Skopiuj tabelę `Address` do swojej bazy danych:

```
select * into address from  adventureworks2017.person.address
```

W tej części będziemy analizować następujące zapytanie:

```
select addressline1, addressline2, city, stateprovinceid, postalcode
from address
```

```
where postalcode between '98000' and '99999'
```

```
create index address_postalcode_1
on address (postalcode)
include (addressline1, addressline2, city, stateprovinceid);
go

create index address_postalcode_2
on address (postalcode, addressline1, addressline2, city, stateprovinceid);
go
```

Czy jest widoczna różnica w planach/kosztach zapytań?

- w sytuacji gdy nie ma indeksów
- przy wykorzystaniu indeksu:
  - address_postalcode_1
  - address_postalcode_2

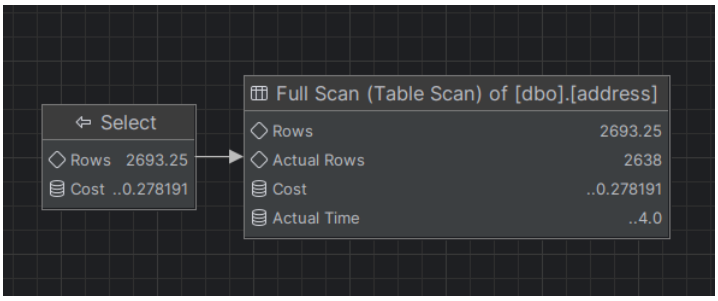
Jeśli tak to jaka?

Aby wymusić użycie indeksu użyj `WITH(INDEX(Address_PostalCode_1))` po `FROM`

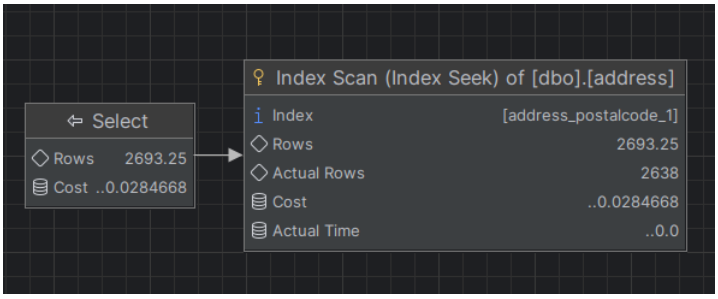
```
select addressline1, addressline2, city, stateprovinceid, postalcode
from address  WITH(INDEX(Address_PostalCode_1))
where postalcode between '98000' and '99999'


select addressline1, addressline2, city, stateprovinceid, postalcode
from address  WITH(INDEX(Address_PostalCode_2))
where postalcode between '98000' and '99999'
```

> Wyniki:

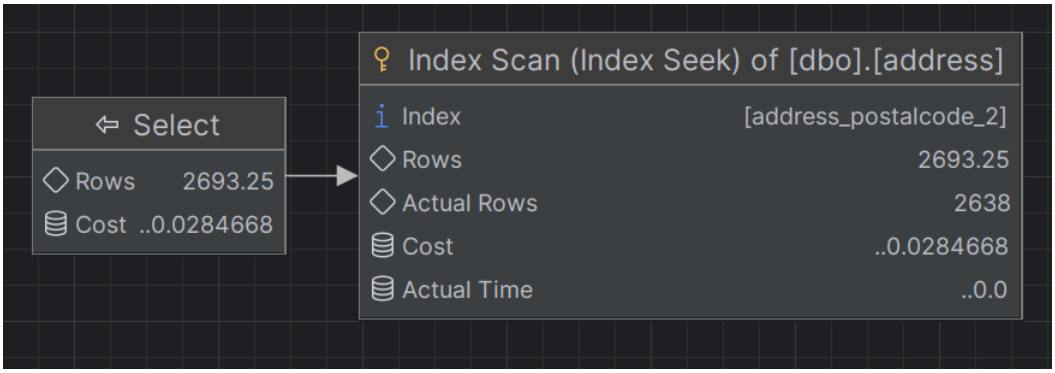
Bez indeksów



address_postalcode_1



address_postalcode_2



Zapytanie	Koszt	Czas (ms)
Bez indeksów	0.278191	4.0
address_postalcode_1	0.0284668	0.0
address_postalcode_2	0.0284668	0.0

W przypadku braku indeksów koszt jest najwyższy, bo serwer musi wykonać Full Table Scan, aby znaleźć pasujące wiersze do warunku.

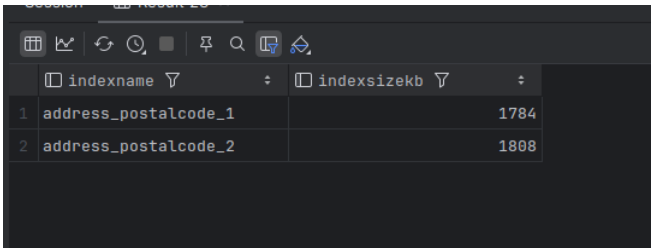
Indeksy address_postalcode_1 i address_postalcode_2 mają te same koszty (niższe niż dla zapytania bez indeksów) i plany. Dla pierwszego serwer przeszukuje indeks po postalcode, a reszta potrzebnych kolumn też znajduje się w indeksie - include. Dla drugiego indeksu wszystkie potrzebne kolumny są częścią indeksu, więc też pokrywają całe zapytanie.

Sprawdź rozmiar Indeksów:

```
select i.name as indexname, sum(s.used_page_count) * 8 as indexsizekb
from sys.dm_db_partition_stats as s
inner join sys.indexes as i on s.object_id = i.object_id and s.index_id = i.index_id
where i.name = 'address_postalcode_1' or i.name = 'address_postalcode_2'
group by i.name
go
```

Który jest większy? Jak można skomentować te dwa podejścia do indeksowania? Które kolumny na to wpływają?

> Wyniki:



	indexname	indexsizekb
1	address_postalcode_1	1784
2	address_postalcode_2	1808

Większy jest address_postalcode_2. Dla niego wszystkie kolumny (addressline1, addressline2, city, stateprovinceid - one wpływają an rozmiar) są częścią klucza indeksu, czyli każda z nich jest przechowywana w strukturze B+ drzewa. W indeksie address_postalcode_1 te same kolumny są w include, czyli są dołączone tylko do liści indeksu, a nie do całej struktury drzewa - przez co indeks ma mniejszy rozmiar.

Zadanie 5 - kolejność atrybutów

Skopiuj tabelę `Person` do swojej bazy danych:

```
select businessentityid
      ,persontype
      ,namestyle
      ,title
      ,firstname
      ,middlename
      ,lastname
      ,suffix
      ,emailpromotion
      ,rowguid
      ,modifieddate
into person
from adventureworks2017.person.person
```

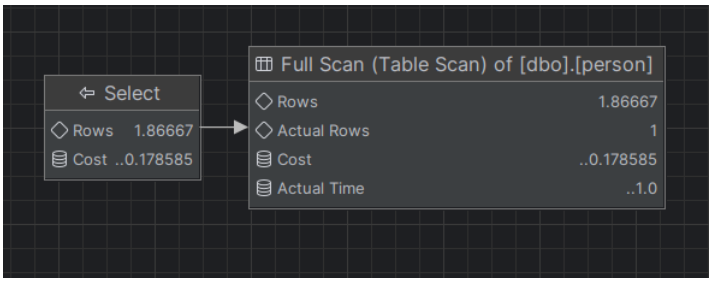
Wykonaj analizę planu dla trzech zapytań:

```
--- 1
select * from [person] where lastname = 'Agbonile'

--- 2
select * from [person] where lastname = 'Agbonile' and firstname = 'Osarumwense'

--- 3
select * from [person] where firstname = 'Osarumwense'
```

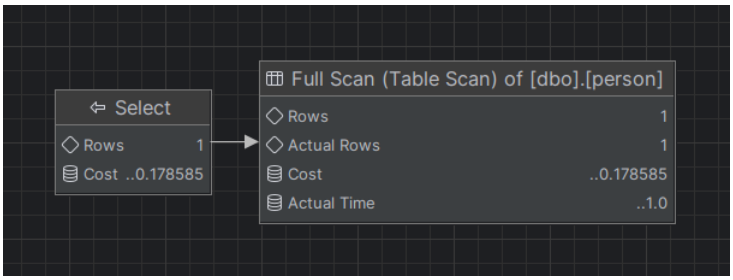
Co można o nich powiedzieć?

> Wyniki:

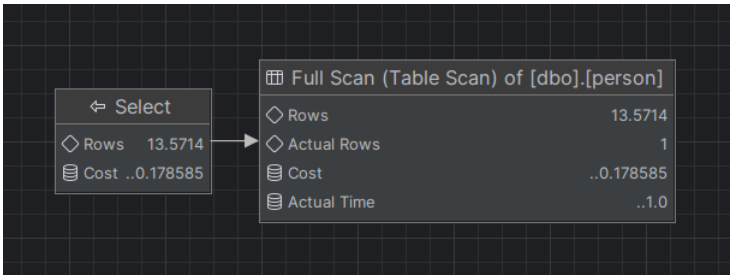
--- 1



--- 2



--- 3



Serwer we wszystkich trzech przypadkach wykonuje Full Table Scan, aby odnaleźć wyniki zapytań.

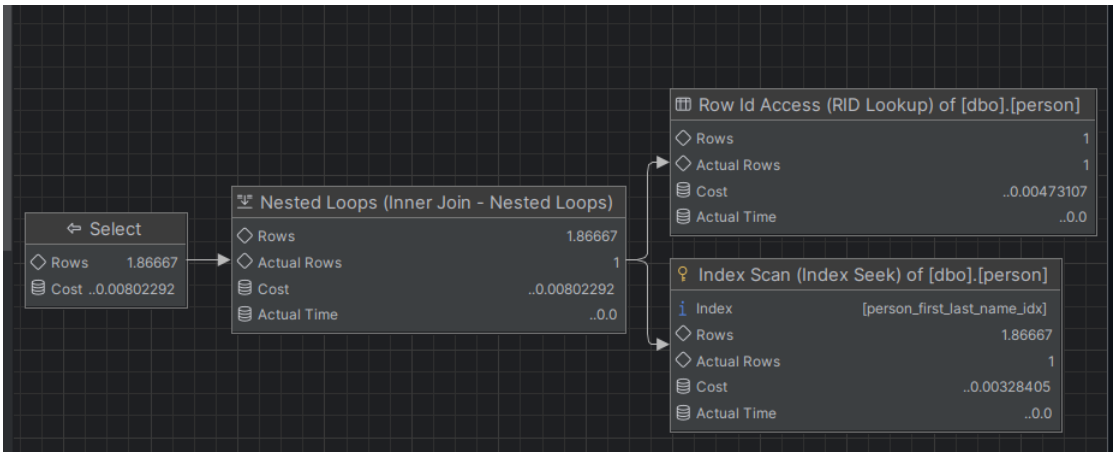
Przygotuj indeks obejmujący te zapytania:

```
create index person_first_last_name_idx
on person(lastname, firstname)
```

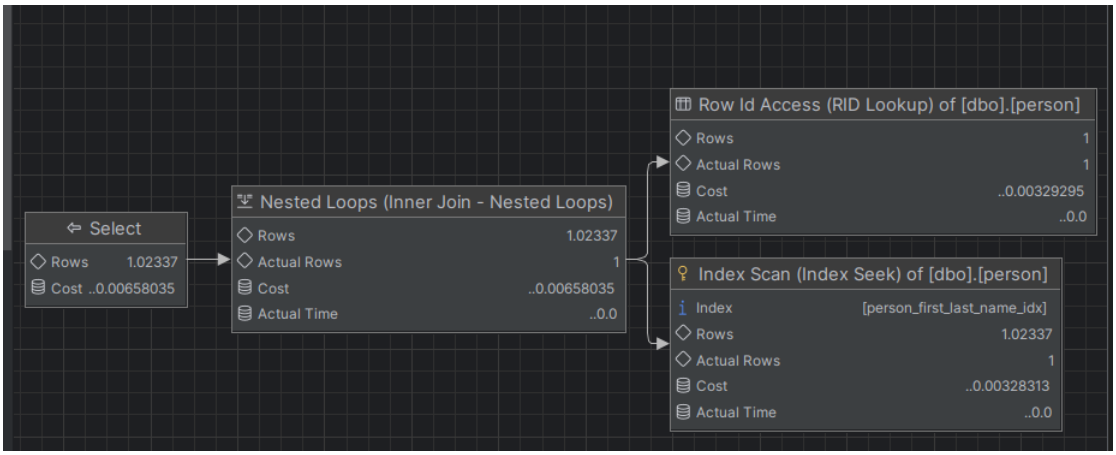
Sprawdź plan zapytania. Co się zmieniło?

> Wyniki:

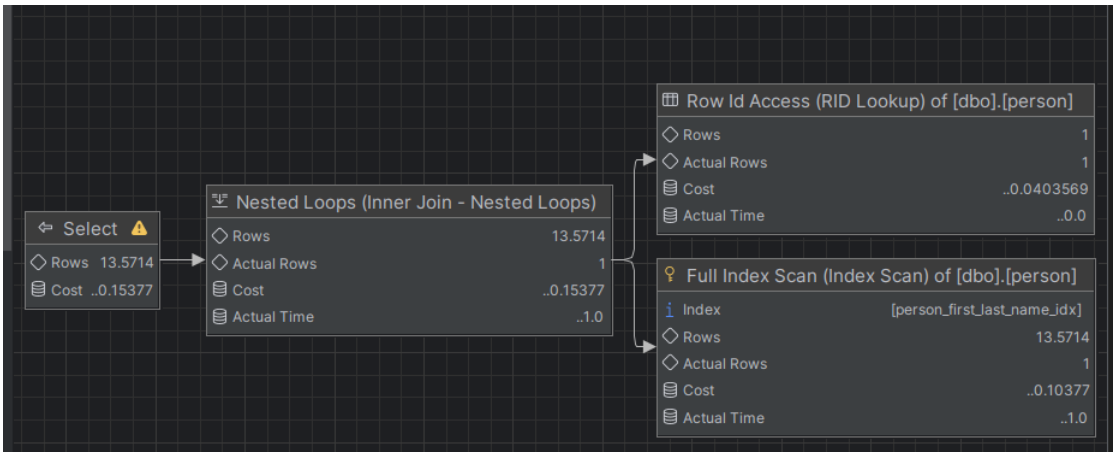
--- 1



--- 2



--- 3



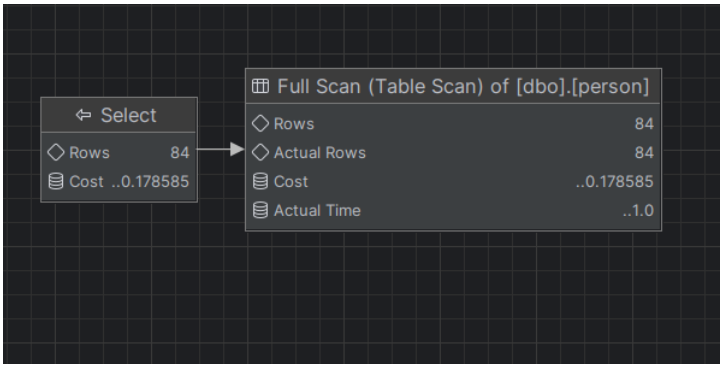
Indeks wielokolumnowy traktuje pierwszą kolumne jako najważniejszą - serwer może korzystać w pełni z indeksu dopóki lastname jest użyte w zapytaniu. W 1 i 2 zapytaniu widać Index Seek (precyzyjne wyszukiwanie w indeksie - serwer wie dokładnie, gdzie są szukane wiersze), a dla 3 mamy tylko Index Scan (przeszukiwanie całego indeksu).

Przeprowadź ponownie analizę zapytań tym razem dla parametrów: `FirstName = ‘Angela’ LastName = ‘Price’`. (Trzy zapytania, różna kombinacja parametrów).

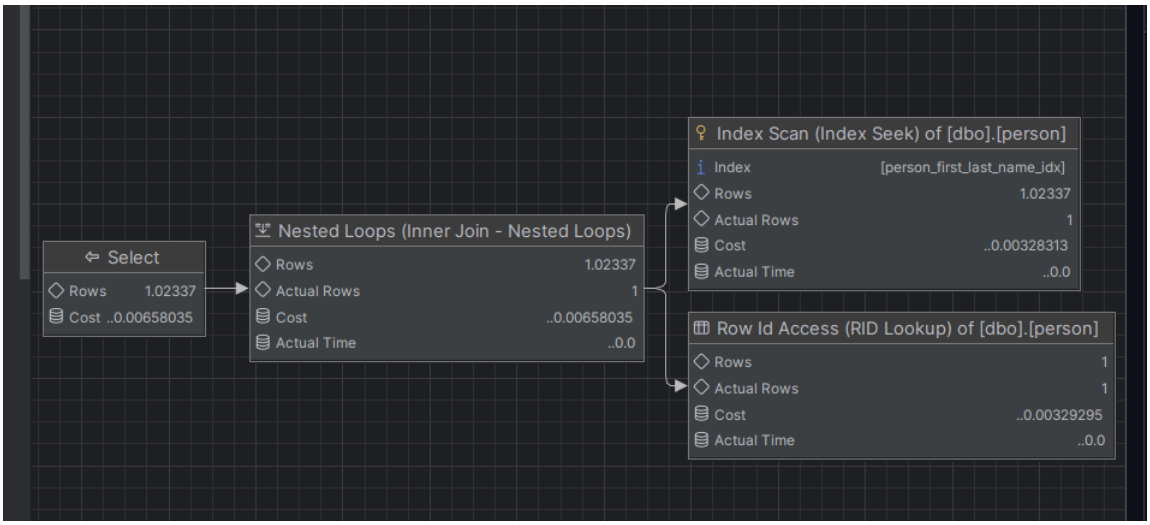
Czym różni się ten plan od zapytania o `'Osarumwense Agbonile'` . Dlaczego tak jest?

> Wyniki:

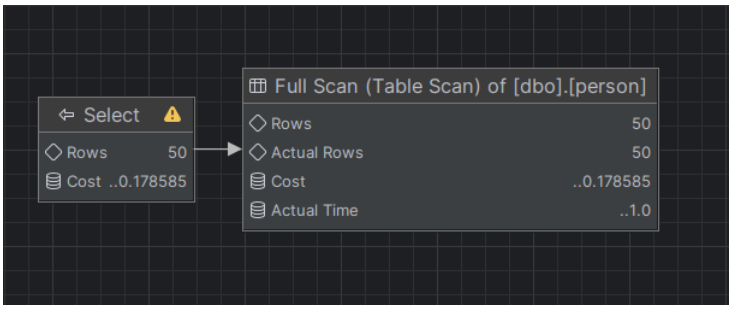
--- 1



--- 2



--- 3



W tym przypadku indeks został wykorzystany tylko dla 2 zapytania (połączenie lastname i firstname). Dla 1 i 3 serwer użył Full Table Scan. Widać na tym przykładzie, że serwer też decyduje jakiego sposobu użyć w zależności od kosztu wyszukania. Dla danych `FirstName = 'Angela' LastName = 'Price'` mamy o wiele więcej rekordów dla pojedyńczych wyszukiwań (tylko lastname albo firstname), a w przypadku `'Osarumwense Agbonile'` istnieje tylko jeden rekord o tym imieniu oraz również jeden o tym nazwisku. Koszt okazał się mniejszy w 1 i 3 zapytaniu (`FirstName = 'Angela' LastName = 'Price'`) dla odczytu całej tabeli, niż skakaniu po indeksie oraz odwoływaniu się do tabeli dla każdego wiersza (select *).

Punktacja:

zadanie	pkt
1	2
2	2
3	2
4	2
5	2
razem	10